NLP · Day 30 of 30 · about 60 minutes

Capstone: review analyzer + what next

🎯 Today's goal

On the last day of PyPath you will:

- build a complete **review analyzer**, step by step, from pieces you already know
- combine text cleaning (Day 27), TF-IDF (Day 28) and the sentiment classifier (Day 29)
- wrap everything in one `analyze()` function that returns a dictionary of results
- turn the results into a printed report and a chart
- look back at 30 days of learning, and plan what to learn next

🤔 Why this matters

Imagine the manager of our made-up online shop. Every Monday she wants to know: *How happy were customers last week? What do happy customers like? What do unhappy customers complain about?* She doesn't want to read every review, and she doesn't want to run ten code boxes. She wants **one call that gives an answer**.

That's what a **capstone** is: a final project that joins everything together. Real data work looks exactly like this. Small, tested pieces (cleaning, a model, a few counts) are combined into a tool that someone else can use with one line.

🏷️ Step 1: the training data

These are the same 80 **made-up** labelled reviews as yesterday. They teach our model what positive and negative reviews look like:

```
# 80 made-up reviews from a made-up online shop (invented for this lesson)
labelled_reviews = [
    ("Great sound and the battery lasts all day", "positive"),
    ("I love these headphones, so comfortable", "positive"),
    ("Fast delivery and excellent quality", "positive"),
    ("Works perfectly, I would buy it again", "positive"),
    ("Excellent value, great price", "positive"),
    ("The kettle is fast and looks lovely", "positive"),
    ("Really happy with this lamp, lovely warm light", "positive"),
    ("Best backpack I have ever owned, love it", "positive"),
    ("Strong and light, great for travel", "positive"),
    ("The blender is powerful and easy to clean", "positive"),
    ("Great phone case, fits perfectly", "positive"),
    ("Comfortable shoes, I wear them every day", "positive"),
    ("Great gift, my sister loved it", "positive"),
    ("Arrived early and works great", "positive"),
    ("Fast charger, highly recommend", "positive"),
    ("Beautiful colour and soft fabric, love it", "positive"),
    ("The keyboard is excellent and quiet", "positive"),
    ("Amazing picture quality, really happy", "positive"),
    ("Easy to set up, the app is simple and good", "positive"),
    ("Good quality for a low price", "positive"),
    ("Not bad at all, does the job well", "positive"),
    ("Really good, better than I expected", "positive"),
    ("Perfect size and well made", "positive"),
    ("The coffee machine is excellent, great taste", "positive"),
    ("Happy with my purchase, nice and sturdy", "positive"),
    ("Lovely mug, keeps my tea hot", "positive"),
    ("The speaker is loud and clear, love it", "positive"),
    ("Five stars, excellent customer service", "positive"),
    ("Cheap but works really well", "positive"),
    ("Exactly as described, very happy", "positive"),
    ("Smooth, quiet and fast, I love it", "positive"),
    ("Great for running, light and comfortable", "positive"),
    ("My kids love this game", "positive"),
    ("Nice design and the battery is great", "positive"),
    ("The best toaster I have had, works perfectly", "positive"),
    ("The watch looks great and works well", "positive"),
    ("Sturdy table, easy to build, recommend", "positive"),
    ("I recommend it to everyone, great buy", "positive"),
    ("The pillow is soft and comfortable", "positive"),
    ("Good sound, good price, happy customer", "positive"),
    ("Terrible quality, it broke after a week", "negative"),
    ("The battery is poor and dies after two hours", "negative"),
    ("Not good, the sound is poor", "negative"),
    ("Arrived late and the box was broken", "negative"),
    ("Waste of money, do not buy", "negative"),
    ("The kettle leaks, terrible", "negative"),
    ("Very disappointed, it stopped working", "negative"),
    ("Cheap plastic, it broke on day one", "negative"),
    ("The app keeps crashing, useless", "negative"),
    ("Too small and uncomfortable, returned it", "negative"),
```

```
    ("Poor quality, the strap broke", "negative"),
    ("The charger is slow and stopped working", "negative"),
    ("Awful customer service, never replied", "negative"),
    ("Not worth the price, disappointed", "negative"),
    ("The colour faded after one wash, poor", "negative"),
    ("Terrible screen, the picture is blurry", "negative"),
    ("Returned it, the worst blender I have used", "negative"),
    ("The shoes are uncomfortable and hurt my feet", "negative"),
    ("Noisy fan and the remote is useless", "negative"),
    ("Broken on arrival, very disappointed", "negative"),
    ("The lamp is awful and the light is cold", "negative"),
    ("Bad smell and the fabric is rough", "negative"),
    ("Hard to set up, the instructions are useless", "negative"),
    ("The speaker is terrible at high volume", "negative"),
    ("I would not recommend this, awful", "negative"),
    ("The mug broke in the dishwasher", "negative"),
    ("Slow delivery and the wrong size", "negative"),
    ("The keyboard is bad and some keys stick", "negative"),
    ("Stopped working after a month, waste of money", "negative"),
    ("Awful taste, the coffee machine leaks", "negative"),
    ("Poor table, it wobbles", "negative"),
    ("The game is boring, a waste", "negative"),
    ("Overpriced and badly made, disappointed", "negative"),
    ("The backpack zip broke, terrible", "negative"),
    ("Disappointed, it looks nothing like the photo", "negative"),
    ("The pillow is too hard and uncomfortable", "negative"),
    ("Not happy, the watch stopped working", "negative"),
    ("Uncomfortable and the sound is poor", "negative"),
    ("The toaster is the worst, returned it", "negative"),
    ("Bad quality, bad price, unhappy customer", "negative"),
]

texts = [text for text, label in labelled_reviews]
labels = [label for text, label in labelled_reviews]
print(len(texts), "labelled reviews:", labels.count("positive"), "positive,",
labels.count("negative"), "negative")
```

```
80 labelled reviews: 40 positive, 40 negative
```

🧠 Step 2: train the final model

Yesterday we kept 20 reviews aside and measured an accuracy of 95% on them. Now that we trust the recipe, we train the final model on **all 80** reviews: more examples means a better model. (This is normal practice: test first, then train the version you use on everything you have.)

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

model = make_pipeline(TfidfVectorizer(), LogisticRegression())
model.fit(texts, labels)

print("Trained on", len(texts), "reviews. Classes:", model.classes_)
```

```
Trained on 80 reviews. Classes: ['negative' 'positive']
```

🧼 Step 3: a cleaning function for keywords

To find out *what* customers talk about, we'll count words, just like on Day 27. This time we borrow scikit-learn's English stop-word list (the one behind `stop_words="english"` on Day 28). It's a ready-made set of 318 words. We don't stem here, because the manager should read real words like *battery*, not *batteri*.

```
import re
from collections import Counter
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

def clean_text(text):
    """Lowercase, find the words, drop stop words."""
    words = re.findall(r"[a-z']+", text.lower())
    return [w for w in words if w not in ENGLISH_STOP_WORDS]

print(len(ENGLISH_STOP_WORDS), "stop words")
print(clean_text("The sound is GREAT and the battery lasts for days!"))
```

```
318 stop words
['sound', 'great', 'battery', 'lasts', 'days']
```

📬 Step 4: this week's new reviews

Here are 12 new **made-up** reviews that arrived this week. The model has never seen them, and they have no labels: that's the model's job. For each review we ask for the label and for the chance that it's positive:

```
import pandas as pd

# 12 new made-up reviews (no labels)
new_batch = [
    "The sound is great and the battery lasts for days",
    "Terrible battery, it dies after two hours",
    "Lovely design, I love the colour",
    "Delivery was slow and the box was broken",
    "Excellent sound, fast delivery",
    "Poor battery and the handle broke",
    "Comfortable and light, great sound",
    "Waste of money, the battery is useless",
    "Really happy, great design",
    "Slow delivery, very disappointed",
    "Great value, the sound is excellent",
    "The screen is blurry and the battery is poor",
]

table = pd.DataFrame({
    "review": new_batch,
    "sentiment": model.predict(new_batch),
    "chance positive": model.predict_proba(new_batch)[:, 1].round(2),
})
table
```

	review	sentiment	chance positive
0	The sound is great and the battery lasts for days	positive	0.57
1	Terrible battery, it dies after two hours	negative	0.38
2	Lovely design, I love the colour	positive	0.62
3	Delivery was slow and the box was broken	negative	0.39
4	Excellent sound, fast delivery	positive	0.66
5	Poor battery and the handle broke	negative	0.32
6	Comfortable and light, great sound	positive	0.72
7	Waste of money, the battery is useless	negative	0.32
8	Really happy, great design	positive	0.75
9	Slow delivery, very disappointed	negative	0.37
10	Great value, the sound is excellent	positive	0.63
11	The screen is blurry and the battery is poor	negative	0.32

`[:, 1]` takes every row of column 1, which is `"positive"` in `model.classes_`.

📊 Step 5: the share of happy customers

```
predicted = list(model.predict(new_batch))

positive_count = predicted.count("positive")
positive_share = positive_count / len(predicted) * 100
print(f"{positive_count} of {len(predicted)} reviews are positive:
{positive_share:.1f}%")
```

```
6 of 12 reviews are positive: 50.0%
```

🔑 Step 6: top keywords per sentiment

Now split the reviews by their predicted label and count the cleaned words in each group. A small helper function keeps this tidy:

```
def top_keywords(reviews, n=3):
    """The n most common cleaned words in a list of reviews."""
    counts = Counter()
    for review in reviews:
        counts.update(clean_text(review))
    return [word for word, count in counts.most_common(n)]

happy = [r for r, label in zip(new_batch, predicted) if label == "positive"]
unhappy = [r for r, label in zip(new_batch, predicted) if label == "negative"]

print("Happy customers mention:  ", top_keywords(happy))
print("Unhappy customers mention:", top_keywords(unhappy))
```

```
Happy customers mention:   ['sound', 'great', 'design']
Unhappy customers mention: ['battery', 'delivery', 'slow']
```

There's the answer for the manager: people love the **sound** and the **design**, and they complain about the **battery** and slow **delivery**.

🧩 Step 7: everything in one `analyze()` function

Now we wrap steps 4–6 into one function (Day 6) that returns a **dictionary** (Day 3) of results. A dictionary is perfect here: each result gets a clear name. If the list is empty we `raise` an error with a helpful message (Day 7) instead of dividing by zero.

```
def analyze(reviews):
    """Predict the sentiment of each review and summarise the batch."""
    if len(reviews) == 0:
        raise ValueError("There are no reviews to analyze.")

    # str() turns NumPy's text values into plain Python text, so they print neatly
    predicted = [str(label) for label in model.predict(reviews)]
    happy = [r for r, label in zip(reviews, predicted) if label == "positive"]
    unhappy = [r for r, label in zip(reviews, predicted) if label == "negative"]

    return {
        "count": len(reviews),
        "positive": len(happy),
        "negative": len(unhappy),
        "positive_share": len(happy) / len(reviews) * 100,
        "labels": predicted,
        "top_positive": top_keywords(happy),
        "top_negative": top_keywords(unhappy),
    }

result = analyze(new_batch)
for key, value in result.items():
    print(f"{key:15} {value}")
```

```
count           12
positive        6
negative        6
positive_share  50.0
labels          ['positive', 'negative', 'positive', 'negative', 'positive',
'negative', 'positive', 'negative', 'positive', 'negative', 'positive',
'negative']
top_positive    ['sound', 'great', 'design']
top_negative    ['battery', 'delivery', 'slow']
```

Dictionary keys must be spelled exactly. Asking for a key that doesn't exist fails:

💡 This example has a mistake on purpose. Run it and read the error.

```
print(result["positive_percent"])
```

⚠️ KeyError

That key or column name doesn't exist. Check the spelling.

```
Traceback (most recent call last):
  line 1, in <module>
    print(result["positive_percent"])
          ~~~~~~^^^^^^^^^^^^^^^^^^^^
KeyError: 'positive_percent'
```

`KeyError: 'positive_percent'`: the key is called `"positive_share"`. When in doubt, `print(result.keys())`.

📝 Step 8: a report and a chart

The manager wants words, not a dictionary. A `report()` function turns the results into neat lines with f-strings (Day 2). `", ".join(list)` glues the words of a list into one text, with a comma and a space between them:

```
def report(result):
    print("===== Weekly review report =====")
    print(f"Reviews analysed:  {result['count']}")
    print(f"Positive:          {result['positive']}
({result['positive_share']:.1f}%)")
    print(f"Negative:          {result['negative']}")
    print(f"Customers like:    {', '.join(result['top_positive'])}")
    print(f"Customers dislike: {', '.join(result['top_negative'])}")

report(result)
```

```
===== Weekly review report =====
Reviews analysed:  12
Positive:          6 (50.0%)
Negative:          6
Customers like:    sound, great, design
Customers dislike: battery, delivery, slow
```

Inside an f-string that uses double quotes, write the dictionary keys with single quotes: `result['count']`. (Python 3.12 and newer also accept double quotes there, but older versions stop with a `SyntaxError`.)

And a picture (Day 13), with two charts side by side:

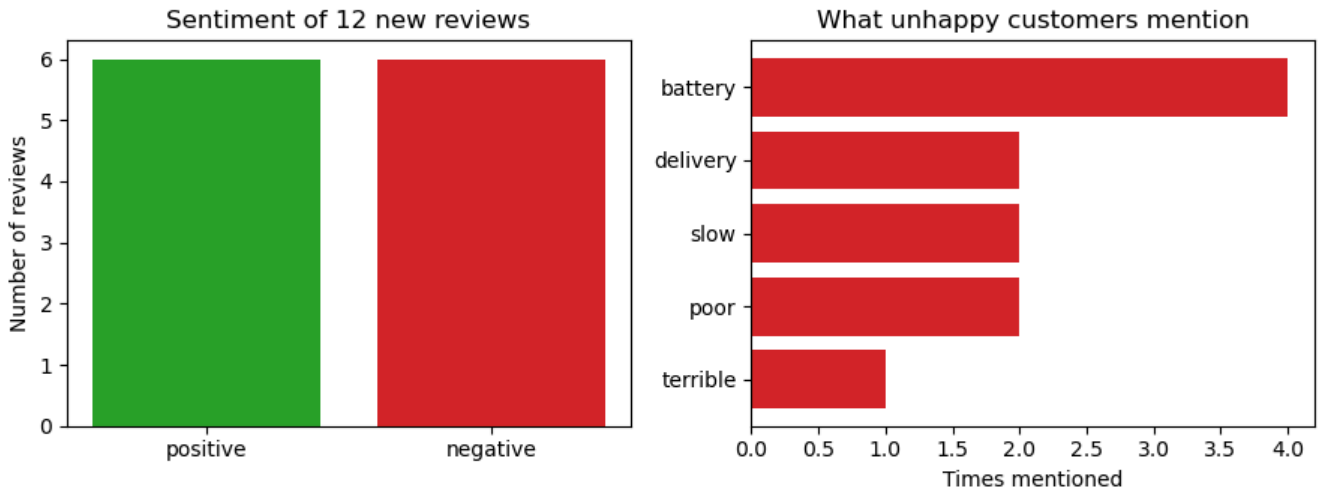
```
import matplotlib.pyplot as plt

fig, (left, right) = plt.subplots(1, 2, figsize=(9, 3.5))

left.bar(["positive", "negative"], [result["positive"], result["negative"]],
         color=["tab:green", "tab:red"])
left.set_title(f"Sentiment of {result['count']} new reviews")
left.set_ylabel("Number of reviews")

unhappy_counts = Counter()
for review, label in zip(new_batch, result["labels"]):
    if label == "negative":
        unhappy_counts.update(clean_text(review))
words, counts = zip(*unhappy_counts.most_common(5))
right.barh(words[::-1], counts[::-1], color="tab:red")
right.set_title("What unhappy customers mention")
right.set_xlabel("Times mentioned")

plt.tight_layout()
plt.show()
```



`zip(*pairs)` does the opposite of `zip`: it turns a list of `(word, count)` pairs into one tuple of words and one tuple of counts.

Our analyzer is done. Try it with your own reviews: change the texts, add more, and run the box again.

```
my_reviews = [
    "I love this keyboard, excellent and quiet",
    "The charger broke after a week, awful",
    "Great price and fast delivery",
]

report(analyze(my_reviews))
```

```
===== Weekly review report =====
Reviews analysed:  3
Positive:          2 (66.7%)
Negative:          1
Customers like:    love, keyboard, excellent
Customers dislike: charger, broke, week
```

Be honest about limits: our model learned from 80 made-up reviews. It will miss sarcasm, "not bad" and words it has never seen (Day 29). Before a real shop relied on it, we would train it on thousands of real, labelled reviews and test it again.

🎓 Looking back: 30 days of PyPath

Thirty days ago, `print("Hello, world!")` was new. Look at what you can do now:

Phase	Days	What you learned
Python Basics	1–8	print and maths, variables and types, lists and dictionaries, if/else, loops, functions, files, modules and errors. Project: an expense tracker.
Data Science	9–15	NumPy arrays, pandas tables (loading, filtering, grouping), cleaning messy data, Matplotlib charts and exploratory data analysis. Project: ship passengers EDA.
Machine Learning	16–22	train/test splits, linear and logistic regression, KNN, decision trees and forests, precision, recall and cross-validation, K-means clustering. Project: a survival predictor.
Deep Learning	23–26	a neuron from scratch, gradient descent, a 2-layer network, recognising handwritten digits, overfitting and regularisation, a look at PyTorch and Keras.
NLP	27–30	cleaning and tokenising text, bag-of-words, TF-IDF, similarity search, a sentiment classifier. Project: today's review analyzer.

The most important skill isn't on the list: you've learned to **read error messages**, break a problem into small steps, test each one, and check your results instead of trusting them blindly. That's what programmers and data scientists do every day.

⚠️ Common mistakes

- Misspelling a dictionary key → `KeyError`. Check with `result.keys()`.
- Dividing by `len(reviews)` when the list is empty → `ZeroDivisionError`. Check first, or `raise` a clear error.
- Using double quotes for a key inside a double-quoted f-string: a `SyntaxError` before Python 3.12. Use `result['count']`, which works everywhere.
- Fitting the model inside `analyze()`. Train once, outside; the function only predicts.
- Trusting the tool more than the data it learned from.

✅ Recap

- A capstone joins small, tested pieces into one tool: clean → vectorise → classify → summarise → report.
- Train the final model on all the labelled data once you've tested the recipe.
- `analyze()` returns a dictionary, so every result has a name.
- `report()` and a chart turn numbers into something a person can act on.
- Know the limits of your model, and say them out loud.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

Exercises 3 and 5 train a small model on the 24 made-up reviews from yesterday's exercises, so the boxes stay short.

1. Count the keywords

Count every word in the reviews that is **not** a stop word, in a `Counter` called `keyword_counts`. Then store the most common keyword in `top_keyword`.

```
from collections import Counter

reviews = ["great sound and great battery", "the price is great", "sound is
poor", "battery died fast"]
stop_words = {"the", "and", "is", "a"}

# Count the keywords, then store top_keyword
```

Hints:

1. Start with `keyword_counts = Counter()`, then loop over the reviews.
2. For each review: `keyword_counts.update([w for w in review.split() if w not in stop_words])`
3. `top_keyword = keyword_counts.most_common(1)[0][0]`

2. The share of happy customers

A model predicted these labels for eight reviews. Store the **percentage** of positive labels (0 to 100) in `positive_share` and print it with one decimal, like: `Positive: 62.5%`

```
predicted = ["positive", "negative", "positive", "positive",
             "negative", "positive", "negative", "positive"]

# Store positive_share, then print it
```

Hints:

1. `predicted.count("positive")` counts the positive labels.
2. `positive_share = predicted.count("positive") / len(predicted) * 100`
3. `print(f"Positive: {positive_share:.1f}%")`

3. A small `analyze()`

Finish `analyze(reviews)`. It should use the trained `model` to predict a label for every review and return a dictionary with these four keys:

- `"count"`: how many reviews there are,
- `"positive"` and `"negative"`: how many got each label,
- `"positive_share"`: the percentage of positive reviews (0 to 100).

The checker will call it with several lists of reviews.

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)
model = make_pipeline(TfidfVectorizer(), LogisticRegression())
model.fit(texts, labels)

def analyze(reviews):
    # predict the labels, then return the dictionary
    return {}

print(analyze(["great sound", "it broke, terrible", "love the colour"]))
```

Hints:

1. Inside the function: `predicted = list(model.predict(reviews))`
2. `positive = predicted.count("positive")`, and the same for `"negative"`.
3. `return {"count": len(reviews), "positive": positive, "negative": negative, "positive_share": positive / len(reviews) * 100}`

4. Report lines

Write `make_report(result)` that takes a result dictionary like the one below and **returns a list of three lines** (texts):

1. `Reviews: 12`
2. `Positive: 58.3%` (one decimal)
3. `Verdict: mostly happy` if the share is 50 or more, otherwise `Verdict: mostly unhappy`

The checker will try it with other dictionaries too.

```
result = {"count": 12, "positive": 7, "negative": 5, "positive_share": 58.333333}

def make_report(result):
    # return a list of three lines
    return []

for line in make_report(result):
    print(line)
```

Hints:

1. First line: `f"Reviews: {result['count']}"`. Second: `f"Positive: {result['positive_share']:.1f}%"`.
2. Choose the verdict with `if result["positive_share"] >= 50:` … `else:` …
3. Finish with `return [line1, line2, line3]`.

5. Challenge: the happiest review

Extend `analyze()` so the dictionary also has a key `"most_positive"`: the review with the **highest chance of being positive** according to `model.predict_proba`. Keep the other keys working.

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)
model = make_pipeline(TfidfVectorizer(), LogisticRegression())
model.fit(texts, labels)

def analyze(reviews):
    predicted = list(model.predict(reviews))
    positive = predicted.count("positive")
    return {
        "count": len(reviews),
        "positive": positive,
        "negative": len(reviews) - positive,
        "positive_share": positive / len(reviews) * 100,
        # add "most_positive" here
    }

print(analyze(["great sound", "it broke, terrible", "excellent, love it"]))
```

Hints:

1. The chance of "positive" for every review: `chances = model.predict_proba(reviews)[:, 1]` (column 1 is "positive" in `model.classes_`).
2. `chances.argmax()` is the position of the biggest chance.
3. Add `"most_positive": reviews[chances.argmax()],` to the dictionary.

🚀 What next

PyPath is finished, but this is a beginning, not an end. Pick one or two of these and keep going. A little every week beats a lot once.

- **Run Python on your own computer.** Install Python from python.org and an editor such as VS Code. Everything you learned works the same way there.
- **Notebooks:** Jupyter works like these pages (code boxes plus notes). Google Colab runs notebooks in the browser for free, even with a GPU.
- **Practise on real data:** Kaggle has thousands of free datasets and beginner competitions, including the real Titanic dataset that inspired our *Northern Star* ship.
- **Learn Git and GitHub** to save every version of your code and share your projects. A few finished projects on GitHub are a great portfolio.
- **Go deeper in pandas** (its official "10 minutes to pandas" guide) and in **statistics**: averages and spread, probability, sampling and hypothesis tests help you trust (or doubt) your results.
- **Deep learning frameworks:** PyTorch or Keras, as previewed on Day 26.
- **Modern NLP:** Hugging Face offers free courses and ready-trained *transformer* models that read words in context, so they handle "not bad" and many sarcastic reviews far better than our bag of words.
- **Build something you care about:** your own expenses, your sports team's results, your study hours. A small real project teaches more than any course.

Thank you for learning with PyPath. Happy coding! 🐍

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Count the keywords

```
from collections import Counter

reviews = ["great sound and great battery", "the price is great", "sound is
poor", "battery died fast"]
stop_words = {"the", "and", "is", "a"}

keyword_counts = Counter()
for review in reviews:
    keyword_counts.update([w for w in review.split() if w not in stop_words])

top_keyword = keyword_counts.most_common(1)[0][0]
print(keyword_counts)
print(top_keyword)
```

2. The share of happy customers

```
predicted = ["positive", "negative", "positive", "positive",
             "negative", "positive", "negative", "positive"]

positive_share = predicted.count("positive") / len(predicted) * 100
print(f"Positive: {positive_share:.1f}%")
```

3. A small analyze()

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)
model = make_pipeline(TfidfVectorizer(), LogisticRegression())
model.fit(texts, labels)

def analyze(reviews):
    predicted = list(model.predict(reviews))
    positive = predicted.count("positive")
    negative = predicted.count("negative")
    return {
        "count": len(reviews),
        "positive": positive,
        "negative": negative,
        "positive_share": positive / len(reviews) * 100,
    }

print(analyze(["great sound", "it broke, terrible", "love the colour"]))
```

4. Report lines

```
result = {"count": 12, "positive": 7, "negative": 5, "positive_share": 58.333333}

def make_report(result):
    line1 = f"Reviews: {result['count']}"
    line2 = f"Positive: {result['positive_share']:.1f}%"
    if result["positive_share"] >= 50:
        line3 = "Verdict: mostly happy"
    else:
        line3 = "Verdict: mostly unhappy"
    return [line1, line2, line3]

for line in make_report(result):
    print(line)
```

5. Challenge: the happiest review

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

positive = ["great sound, love it", "excellent quality and fast delivery",
            "works perfectly, great price", "really happy, highly recommend",
            "comfortable and well made", "love the colour, great buy",
            "fast and easy to use", "excellent, would buy again",
            "good value, happy customer", "great battery and lovely design",
            "perfect fit, very comfortable", "easy to set up, works great"]
negative = ["terrible, it broke after a week", "poor quality, waste of money",
            "awful sound, very disappointed", "stopped working, returned it",
            "slow delivery and a broken box", "uncomfortable and too small",
            "bad battery, dies quickly", "useless app, keeps crashing",
            "poor fit, disappointed", "terrible customer service",
            "broke on day one, awful", "waste of time, bad quality"]
texts = positive + negative
labels = ["positive"] * len(positive) + ["negative"] * len(negative)
model = make_pipeline(TfidfVectorizer(), LogisticRegression())
model.fit(texts, labels)

def analyze(reviews):
    predicted = list(model.predict(reviews))
    positive = predicted.count("positive")
    chances = model.predict_proba(reviews)[:, 1]
    return {
        "count": len(reviews),
        "positive": positive,
        "negative": len(reviews) - positive,
        "positive_share": positive / len(reviews) * 100,
        "most_positive": reviews[chances.argmax()],
    }

print(analyze(["great sound", "it broke, terrible", "excellent, love it"]))
```